Exercise Sheet 2

Answer Key

Information Security Lab: Module 2 • October 28, 2026

This exercise is **ungraded**.
Please think about these exercises *before* the session. We will discuss them together in class.

Exercise 1: Who Stops This Attack?

For each scenario, name the **adversary path** (software, DMA, physical) and the **mechanism** that is usually employed by TEEs to stop it (access control, encryption, integrity, freshness). Then say whether **SGX (client)** and **SEV-SNP** stop it.

1. The hypervisor maps an enclave/CVM page into its own address space and reads it.
2. An admin freezes a DIMM and reads it in another machine (cold boot attack).
3. A malicious device writes via DMA into a protected page.
4. A bus interposer records the ciphertext of a counter and writes it back later.
5. The hypervisor swaps the physical pages behind two guest-physical addresses of a CVM.
6. Code in a co-located VM hammers DRAM rows and flips a bit inside a protected page.

Solution

#	Path	Mechanism needed	SGX (client)	SEV-SNP
1	Software	Access control / Encryption	Blocked by access control	Read returns ciphertext only
2	Physical	Encryption[†]	Protected	Protected
3	DMA	Access control on the DMA path	Blocked	Blocked
4	Physical	Freshness	Detected[*]	Not detected
5	Software	Mapping integrity	Blocked by CPU	Detected via guest page validation
6	Software causing physical fault	Integrity	Detected; machine locks (integrity kept, availability lost)	Silent corruption

[*] Only SGX (client). Scalable SGX dropped the integrity tree for performance, so it does not detect replay. Freshness can be considered as part of integrity.

[†] Cold boot attacks: DRAM keeps its contents for a short time after power is lost, and cooling the DIMM makes that time much longer. An attacker can freeze the DIMM, move it to another machine and read out its contents. For details, see Halderman et al., *Lest We Remember: Cold-Boot Attacks on Encryption Keys*, USENIX Security 2008. Both TEEs stop this attack, because DRAM only ever holds ciphertext and the key never leaves the CPU package.

Exercise 2: The Heracles Byte-by-Byte Attack

Setting

A confidential VM's (CVM) memory is encrypted by the hardware in **4-byte blocks** (16 bytes in reality). A block with plaintext P at physical address $addr$ is stored as

$$C = E_K(P \oplus T(addr)) \oplus T(addr),$$

where E is a deterministic encryption algorithm, K is the secret encryption key and $T(addr)$ the tweak which depends only on the address and never changes.
The attacker is the malicious hypervisor. It does not know K and therefore *cannot* decrypt, but it can:

- **Read** the ciphertext of any CVM memory.
- **Move** a CVM page to an address it chooses. The page is automatically re-encrypted with the tweak of the new address, and the CVM does not notice.
- **Pause** the CVM at any point.
- **Inject** data it chooses into a CVM page (e.g., via network packets).

Task 1. Why does the tweak exist? If the same value is written to the *same* address twice, what do the two ciphertexts look like?

Solution

The tweak makes equal values stored at *different* addresses encrypt differently. At the *same* address, the ciphertext is identical both times: encryption is deterministic per address.

Task 2. A CVM stores a secret value S at address A on an encrypted block of a page. S uses all 4 bytes of the block. Use the hypervisor capabilities above to build a test with which the hypervisor can check: *"Is S equal to my guess g?"*.

Solution

1. Snapshot the ciphertext at A.
2. Move the victim page away from A.
3. Inject g into a block at the same offset in another page, then move that page to A (it is now encrypted with A's tweak).
4. Compare the new ciphertext with the snapshot. A match means $S = g$.

This is a **chosen-plaintext oracle**.

Task 3. How many guesses does this test need in the worst case if S is completely unknown and the block size is 4 bytes? What about 16 bytes? And what if only **one byte** of S is unknown?

Solution

2^{32} guesses for 4 unknown bytes and 2^{128} for 16, which is infeasible. With only one unknown byte, it is just **256**.

Task 4. A user types a password into a buffer at address A that starts as `00 00 00 00`. Each keystroke writes one character. The attacker pauses the VM after every keystroke. The alphabet is {`a`, `e`, `s`, `t`}. Each round, the attacker snapshots the block's ciphertext every time it changes and then tests guesses for the **new** character:

Round	Snapshot	new = `a`	new = `e`	new = `s`	new = `t`
1	`9E07C4A1`	`41D28F3B`	`7AB31E96`	`E610D25F`	`9E07C4A1`
2	`2C58A70E`	`D90F6B24`	`2C58A70E`	`B7A1093C`	`6E3CF581`
3	`F4135D8B`	`F4135D8B`	`0B9DE247`	`57E2B6A0`	`A86C3F19`

How can the hypervisor recover the password? How many tries does it need in the worst case to recover the password?

Solution

The password is `tea`. In each round, the attacker compares the snapshot with the ciphertexts of its guesses and picks the one that matches:

- **Round 1:** the guess `[t 00 00 00]` matches the snapshot `9E07C4A1` ⇒ first character `t`.
- **Round 2:** the guess `[t e 00 00]` matches `2C58A70E` ⇒ second character `e`.
- **Round 3:** the guess `[t e a 00]` matches `F4135D8B` ⇒ third character `a`.

Cost of the attack:

- **With the alphabet {`a`, `e`, `s`, `t`}:** byte-by-byte, each of the 3 rounds needs at most 4 guesses, so $3 \cdot 4 = 12$ guesses in total. Guessing all three characters at once would need $4 \cdot 4 \cdot 4 = 4^3 = 64$ guesses.
- **With arbitrary bytes and a real 16-byte block:** byte-by-byte, each of the 16 rounds needs at most 256 guesses, so $16 \cdot 256 = 4096$ guesses. Guessing the whole block at once would need $256^{16} = 2^{128}$ guesses, which is infeasible.

Learning the secret one byte at a time turns an **exponential** cost into a **linear** one.
Note: technically the last guess can be inferred, so the worst case of guesses is $(4-1)^3$ or $(256-1)^{16}$ respectively.

Task 5. What happens if the attacker misses a keystroke? Name one defense that stops the attack.

Solution

If the attacker misses a keystroke, the next snapshot contains *two* new, unknown characters instead of one. It must then guess both at the same time:

- **With the alphabet {`a`, `e`, `s`, `t`}:** each unknown character has 4 possible values, so two unknown characters give $4 \cdot 4 = 4^2 = 16$ guesses instead of 4.
  Example: the attacker misses the `e` in `tea` and next sees `[t e a 00]`, so it must try all 16 combinations of `[t ? ? 00]`.
- **With arbitrary bytes (256 possible values):** two unknown bytes give $256 \cdot 256 = 256^2 = 65{,}536$ guesses instead of 256.

In general, every missed keystroke **multiplies** the cost of the next round by the number of possible values: by 4 with the alphabet, and by 256 with arbitrary bytes. After k missed keystrokes, the next round costs 4^{k+1} or 256^{k+1} guesses. The cost grows exponentially, so the attacker wants to catch as many writes as possible.»
Defenses: forbid the hypervisor from moving pages (AMD did something similar in a firmware update), or hide ciphertext from the hypervisor. A stronger AES key does **not** help.

Exercise 3: DMA Attacks

Setting

Peripherals such as network cards (NICs) and GPUs use **DMA** (Direct Memory Access): they read and write main memory directly, without going through the CPU. To limit what a device can touch, the system has an **IOMMU**:

- The hypervisor manages an **I/O page table** for each device. The table maps device addresses (IOVAs) to physical pages, with read/write permissions.
- The IOMMU checks every DMA request against the device's I/O page table.

Checking every DMA against the IOMMU page table is slow. PCIe therefore offers **ATS** (Address Translation Services):

- A device asks the IOMMU for a translation (IOVA → physical address) and caches the answer in its **own on-device translation cache (IOTLB)**.
- From then on, the device sends requests that are marked **"already translated"** and use the physical address. The IOMMU lets these through without checking the page table again.
- When the hypervisor unmaps a page, it removes the entry in the I/O page-table and **immediately** sends the device an **invalidation request** ("drop your translation for this IOVA"). The device answers with an **invalidation completion**.

Attacker: a malicious device plugged into a PCIe port. The device can send any DMA request it likes and ignore any protocol rules.

Task 1. We assume the hypervisor, which configures the IOMMU, is trusted in this task. The device reports that it supports ATS, and the hypervisor turns ATS on.

(a) The hypervisor has freed a page P that was previously used by the device, which is now reused elsewhere in the kernel. List every place where a translation for P existed before the unmap. Which of them does the hypervisor control directly, and which does it not?

(b) The ATS invalidation request was sent to the device when the page P was freed. Is the device now guaranteed to have lost access to P?

(c) Read the rules for "already translated" requests again. Does a malicious device actually need a real translation to access a page?

(d) How would you fix the attack while keeping the device usable?

Solution

(a) The translation existed in the **I/O page table** and the IOMMU's own IOTLB, which the hypervisor controls and clears itself, and in the **device's IOTLB**, which lives on the device. The hypervisor can only *ask* the device to clear it.

(b) No. Invalidation depends on the device cooperating. A malicious device sends the completion (or doesn't) and simply **keeps using its cached translation**. P has been reused, so the device can read and write whatever the kernel now stores there. Sending the invalidation quickly does not help: the revocation is only as trustworthy as the device.

(c) No. The IOMMU does not check "already translated" requests, so the device can put

any physical address in them. That gives it **unrestricted access to memory**: a full bypass of the IOMMU.

(d) **Fix:** never enable ATS for untrusted devices, and block "already translated" requests from them (e.g., with PCIe ACS). The IOMMU then checks every access against the page table, and unmapping really revokes access. A device's IOTLB is only safe if the device itself is trusted (e.g., attested).

Task 2. Now think of a confidential VM (CVM) with a passed-through device and an untrusted hypervisor. Who configures the IOMMU and decides whether ATS is enabled? Can the CVM rely on the IOMMU to keep the device away from its memory? What would you need instead?

Solution

The **untrusted hypervisor** configures the IOMMU and ATS. It could map CVM memory for any device, or let the device bypass the IOMMU via ATS, so the IOMMU protects nothing *for the CVM*.

What's needed instead:

- Hardware checks every DMA request against which memory belongs to the CVM, even translated ones (e.g., SEV-SNP checks DMA against its page-ownership table). Devices can then reach only memory the CVM has explicitly **shared**, such as bounce buffers.
- Devices that the CVM should trust directly need attestation and a secure connection (e.g., AMD SEV-TIO).